
Obligatorio 2026

Contexto

La empresa MacrosUM se encuentra desarrollando un nuevo sistema operativo monotarea denominado Doors. En este contexto, se le encomienda el diseño e implementación del módulo encargado de la administración de procesos y del registro (log) de los eventos generados por los procesos de los usuarios del sistema.

Gran parte de las funcionalidades del sistema deberán ejecutarse mediante comandos de consola, los cuales serán detallados más adelante. Asimismo, se proporcionará un conjunto de archivos .csv con información de prueba que deberán utilizarse para la carga automática de procesos en el sistema.

El sistema deberá ser capaz de administrar procesos pertenecientes a distintos usuarios. Cada usuario deberá identificarse mediante un UID (user ID) numérico único y un alias utilizado dentro del sistema. Además, existirán dos tipos de usuarios: GENERIC y ADMIN.

Cada proceso deberá identificarse mediante un PID (process ID) único y almacenar información relativa a su nombre, usuario propietario, prioridad, estado y conjunto de eventos asociados. Los estados posibles de un proceso serán NEW, PENDING, RUNNING y FINISHED.

Cada proceso estará compuesto por un conjunto de eventos. Los eventos podrán ser de tipo CPU, RAM o DISK, y cada uno de ellos deberá contener una o más instrucciones representadas mediante cadenas de texto (String).

Cuando un proceso sea incorporado al sistema, su estado inicial será NEW. En esta instancia, el proceso deberá almacenarse en una cola de nuevos procesos, sin que aún se haya calculado su prioridad. Los procesos únicamente podrán ser cargados a partir de los archivos de prueba proporcionados a través del comando **pload -p [path con archivo csv de procesos] -u [path con archivo csv de usuarios]**

Posteriormente, mediante el comando: **pprepate**, el sistema deberá tomar los procesos almacenados en la cola de nuevos procesos, calcular su prioridad y moverlos a la estructura de procesos pendientes. Los procesos pendientes deberán mantenerse ordenados por prioridad, de forma que aquel con **mayor prioridad** (calculada como $P(p)$) sea el próximo en ejecutarse.

Cada vez que un proceso pase del estado NEW al estado PENDING, el sistema deberá registrar dicho evento en el log utilizando un formato similar al siguiente:

[2026-03-01 15:23:36]: NEW PENDING PROCESS: PID=123 | notepad.exe | USER:admin UID:525 | P=8541

La prioridad $P(p)$ de un proceso p deberá calcularse utilizando la siguiente fórmula:

$$P(p) = \left(\frac{8N_{CPU}(p.events) + 2N_{RAM}(p.events) + 2N_{DISK}(p.events)}{N_{events}(p.events)} \right) + W(p.user) \cdot N_{events}(p.events)$$
$$W(u) = \begin{cases} 32, & \text{si } u = \text{ADMIN} \\ 16, & \text{si } u = \text{GENERIC} \end{cases}$$

Donde:

- $P(p)$: prioridad del proceso p como un entero (int).
- $N_{CPU}(p.events)$: cantidad de eventos de tipo CPU del proceso p .
- $N_{RAM}(p.events)$: cantidad de eventos asociados a memoria RAM.
- $N_{DISK}(p.events)$: cantidad de eventos asociados a disco.
- $N_{events}(p.events)$: cantidad total de eventos del proceso p .
- $W(p.user)$: es el peso del tipo de usuario del evento (32 si el usuario asociado al evento es ADMIN, 16 si el usuario es GENERIC).

Los procesos pendientes deberán ejecutarse mediante el comando **pexecute**. Dado que Doors es un sistema operativo monotarea, **únicamente podrá existir un proceso en ejecución simultáneamente**. Al comenzar la ejecución de un proceso, el sistema deberá registrar en el log la información correspondiente al proceso y a cada uno de sus eventos. Un ejemplo de salida esperada sería el siguiente:

```
[2026-03-01 15:33:36]: EXECUTING PROCESS: PID=123 | USER:alias UID:321
EVENT: CPU | Instructions [sum, jump, add, subs]
EVENT: RAM | Instructions [read, read, write, read, write]
EVENT: CPU | Instructions [add, subs, mod]
EVENT: DISK | Instructions [seek, write]
```

Asimismo, el sistema deberá permitir finalizar procesos mediante el comando **pfinish** con las siguientes variantes:

```
pfinish OK
pfinish ERROR
pfinish TERM [UID]
```

Los tipos de finalización posibles serán OK, ERROR y TERMINATED. El estado TERMINATED únicamente podrá utilizarse cuando la finalización del proceso sea forzada por un usuario específico. En dicho caso, el comando deberá recibir el UID del usuario responsable de la terminación. Toda finalización deberá registrarse en el log del sistema. Algunos ejemplos de salida esperada son los siguientes:

```
[2026-03-01 16:33:36]: ENDING PROCESS: PID=123 | STATE: OK
```

```
[2026-03-01 16:33:36]: ENDING PROCESS: PID=123 | STATE: TERMINATED by USER:alias UID:65
```

A medida que los procesos finalicen, deberán almacenarse en memoria dentro de una pila de procesos finalizados. La cantidad máxima de procesos almacenados estará determinada por una constante definida en el sistema. Cuando dicha pila alcance su capacidad máxima y sea necesario incorporar un nuevo proceso finalizado, el sistema deberá registrar en el log la información correspondiente a todos los procesos contenidos en la pila, respetando el orden inverso al de finalización. Por ejemplo, suponiendo una capacidad máxima de tres procesos:

```
[2026-03-01 17:33:36]: Finished process stack overflow
```

```
PID=323 notepad.exe | STATE: OK | USER:alias UID:321
```

```
PID=123 config.exe | STATE: TERMINATED | USER:alias UID:321
```

```
PID=321 explorer.exe | STATE: ERROR | USER:alias UID:997
```

Por otra parte, el sistema deberá implementar una funcionalidad de consulta de estado mediante el comando **pstatus**. Este comando deberá permitir visualizar por pantalla el estado actual de los procesos cargados en memoria. Cuando se ejecute sin parámetros adicionales, deberá mostrar el proceso actualmente en ejecución, los procesos pendientes y los últimos procesos finalizados almacenados en memoria. Un ejemplo de salida esperada es el siguiente:

PROCESS STATUS

EXECUTING:

```
PID=456 | notepad.exe | USER:admin UID:525 | P=952
```

PENDING:

```
PID=654 | notepad.exe | USER:admin UID:525 | P=785
```

```
PID=741 | notepad.exe | USER:admin UID:525 | P=541
```

FINISHED:

```
PID=123 ipconfig.exe | STATE: TERMINATED | USER:alias UID:321
```

```
PID=321 dir.exe | STATE: ERROR | USER:alias UID:997
```

```
PID=999 init.exe | STATE: OK | USER:alias UID:997
```

Asimismo, el comando deberá soportar distintas variantes. La opción **pstatus -verbose** deberá incluir el detalle completo de los eventos asociados a cada proceso. Por su parte, la opción **pstatus -u [UID]** deberá mostrar únicamente los procesos pertenecientes al usuario indicado, mientras que **pstatus -p [PID]** deberá presentar el detalle completo de un proceso específico y de todos sus eventos asociados.

Todas las consultas deberán realizarse únicamente sobre los procesos actualmente cargados en memoria. Los procesos descartados previamente de la pila de finalizados no deberán considerarse para estas operaciones.

En cuanto al log, se debe generar un archivo con el siguiente siguiente nombre / formato: DOORS_PROCESS_LOG_FECHA donde la fecha está en formato yyyy-MM-dd y en el log el timestamp debe estar en formato yyyy-MM-dd HH:mm:ss.

El equipo de desarrollo de MacrosUM le facilitará un proyecto java con un gestor de comandos y la interfaz del administrador de procesos así como una librería de TADs habilitados para usar. Recuerde que al tratarse del desarrollo de un sistema operativo, el diseño de las soluciones deben ser lo más eficaces y eficientes tanto en memoria como en órdenes de tiempo de ejecución.

Documentación

La empresa le exige la entrega de un informe en pdf donde se exponga un diagrama UML con el diseño de la solución (relación entre entidades) y un análisis de los algoritmos de las funciones argumentando las decisiones del diseño.

El informe deberá contener:

- Diagrama UML describiendo las relaciones entre entidades
- Análisis y justificación de la elección de los TADs y el diseño del modelo.
- Análisis de órdenes de tiempo de ejecución para cada función.

Entregables

- Proyecto java con la implementación.
- Informe

Consideraciones generales

- El proyecto se debe llamar “obligatorioPII_grupoX”, donde X debe sustituirse con el número de grupo asignado.
- Se debe utilizar los TADs que se proporcionan en el proyecto y NO es posible en esta instancia utilizar TADs de terceros (brindados por la máquina virtual de Java u otros).
- Se debe crear un repositorio GIT para la entrega del obligatorio. Se verificará que todos los participantes hayan realizado commits sobre los proyectos. Se va a penalizar en caso de que esto no suceda.
- Se deberá elegir la estructura más adecuada para resolver cada problema.

-
- No se admiten estructuras intermedias que almacenen los resultados de las consultas. Se deben realizar los cálculos en el momento en que se solicita ejecutar una consulta.
 - La calificación final del trabajo se divide:
 - 65% Informe
 - 35% Proyecto Java
 - Fecha de entrega: Viernes 19 de junio hasta las 23:59. Se debe entregar vía moodle un zip que contenga:
 - Código fuente de la solución.
 - Informe
 - Documento de texto GRUPOXX.txt con los nombres y apellidos de los integrantes del equipo.